

Homework IB - Hate Speech Detection

Aur Marina Iuliana, 1809715

Multilingual Natural Language Processing (MNLP) course
A.Y. 2023-2024

1 Introduction

Hate Speech Detection is a binary NLP classification task designed to detect if a given text contains hateful language towards an individual or a group of people based on prejudices related to *ethnicity*, *gender*, *religion* and other personal characteristics. Specifically, given a sentence s , the aim of the task is to analyze its content and determine if it contains **hate speech** (label 1) or if it contains **neutral speech** (label 0).

2 Dataset Format

From the provided EVALITA dataset, only the files related to the **task A** should be considered. Specifically, the train and test files are extracted from Development Set and Test Set folders respectively in .tsv format. An example of the files structure is provided in Listing 1 to enhance comprehension. The validation set is obtained by splitting **10% of the train set** and using `random state = 42` to ensure reproducibility. The final dimensions for train, validation and test sets are respectively 6155, 684 and 1763 sentences.

3 Model Architecture

Bidirectional Long Short-Term Memory (BiLSTM) model is chosen for the task due to its ability to understand the context of a word considering both the words that precede and succeed it. A description of the model architecture is provided in Listing 2 and Figure 3. Some variations of this architecture were experimented, such as adding L2 regularization, but they made the model too complex and increased the risk of overfitting. The model is trained using the **Adam optimizer** and the **Cross Entropy Loss criterion**, as they represent a common choice for classification tasks.

4 Exploratory data analysis (EDA)

An exploratory data analysis (EDA) was conducted to gain insights into the data. Figure 4 provides a visual representation of the most common neutral and hate words in the text. Analyzing them, it was evident that the majority of the words are just common words (e.g., 'la', 'li', 'la', 'che'). Despite this, some of them (e.g., 'user', 'URL', 'migranti', 'rom'), still bring important information within the context of the text, so it was decided to import an external file containing **120 most common italian words** and use it for the preprocessing part.

Figure 1 offers insights into label frequency and distribution of sentence lengths in the training set, showing a **few dominance of class 1 (hate speech)** over class 0 (neutral speech) and an **average sentence length of around 100 words**. (Note: the same figures for test and validation sets can be found in the notebook).

5 Data Preparation

The data is initially converted into JSONL format and then transformed into HaSpeedeDataset objects, enabling its utilization by the BiLSTM model. Additionally, a basic tokenizer is utilized to replace each word in the train set with its **corresponding index** and to generate the **vocabulary** from all words in the train set, resulting in 17570 words.

6 Baselines

6.1 Random baseline

Dummy Classifier is chosen as the random baseline for the task due to its very simple structure. Specifically, the **stratified strategy** randomly chooses class labels based on the distribution of classes in the training set.

6.2 Sophisticated baseline

A simple **Recurrent Neural Network (RNN)** model is chosen as the sophisticated baseline.

A description of the model architecture is provided in Listing 3. The model is trained using the **Adam optimizer** and the **Cross Entropy Loss criterion**, as they represent a common choice for classification tasks.

7 Hyperparameters Optimization

To find the optimal combination of parameters for the task, several variations of the same parameters were experimented to evaluate their impact on overall performance. Some variations were just manually experimented due to time and computational power constraints. Specifically, the focus was limited on the following parameters:

- **preprocessing** with options True or False
- **batch size** with options 64, 128 or 256
- **hidden dimension** with options 128 or 256
- **learning rate** with options 0.01 or 0.001

The optimal combination of parameters was found to be applying the preprocessing techniques (removing common words and punctuation), a batch size of 64, a hidden dimension of 256 and a learning rate of 0.01.

8 Results

Since we are dealing with small data, overfitting was limited by implementing the **early stopping strategy** with tolerance of three consecutive epochs and choosing a dropout of 0.3.

In Figure 5 and Figure 6, we can observe validation and train sets behaviour over 20 epochs using BiLSTM model. Some few discrepancy between the train and validation losses can be attributed to the use of dropout during the train but not during the validation and the limited data available.

A summary of the models performance can be observed Table 1, which includes metrics such as accuracy, recall, precision, and f1 score, and Figure 2, which includes the models confusion matrices. Upon analyzing the results, we can conclude that BiLSTM model outperforms both Dummy Classifier and RNN model, achieving an **accuracy of 0.72** and a **loss of 0.6**.

9 Code Instructions

The script was run on Google Colab, while the dataset and the common words file were uploaded on Google Drive to make them accessible in the notebook. GPU is necessary to run models and avoid execution time problems.

10 Tables and Figures

Listing 1: Example of one sample from Test Set

```
id , full_text , hs , stereotype
61,"Corriere: Tangenti , Mafia Capitale dimenticataMazzette su buche
e campi rom URL #roma ",0,0
```

Listing 2: BiLSTM Architecture

```
BiLSTMModel(
  (embedding): Embedding(17570, 256, padding_idx=0)
  (bilstm): LSTM(256, 256, num_layers=2, batch_first=True, dropout=0.3,
    bidirectional=True)
  (dropout): Dropout(p=0.3, inplace=False)
  (batchnorm): BatchNorm1d(512, eps=1e-05, momentum=0.1, affine=True,
    track_running_stats=True)
  (projection): Linear(in_features=512, out_features=2, bias=True)
  (sigmoid): Sigmoid()
)
```

Listing 3: RNN Architecture

```
RNNModel(
  (embedding): Embedding(17570, 128, padding_idx=0)
  (rnn): RNN(128, 128, batch_first=True)
  (projection): Linear(in_features=128, out_features=2, bias=True)
)
```

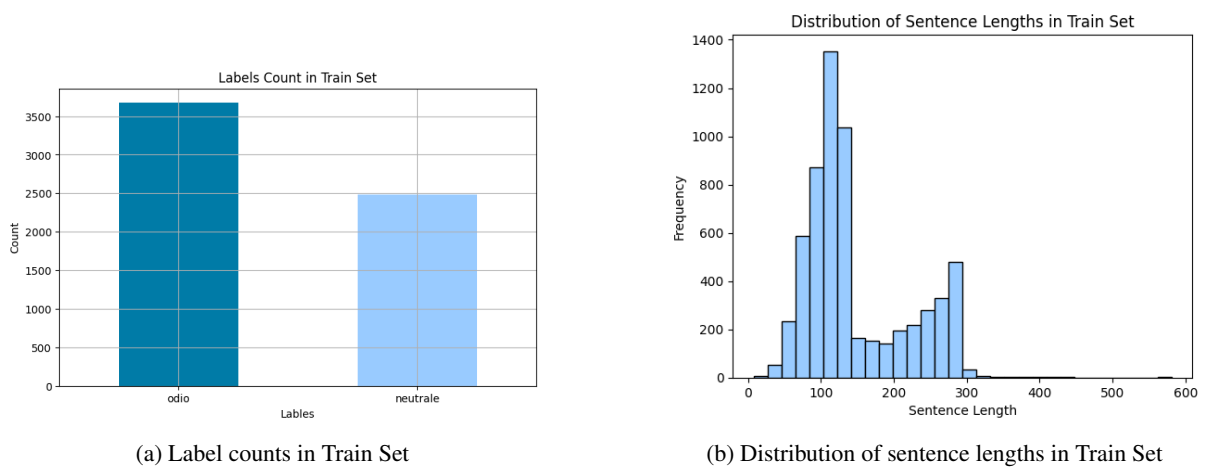


Figure 1: Data insights

Model	Accuracy	Precision	Recall	F1-score
Dummy Classifier	0.5	0.44	0.39	0.42
BiLSTM	0.72	0.93	0.72	0.8
RNN	0.66	0.92	0.66	0.76

Table 1: Performance Comparison between Dummy Classifier, BiLSTM model and RNN model

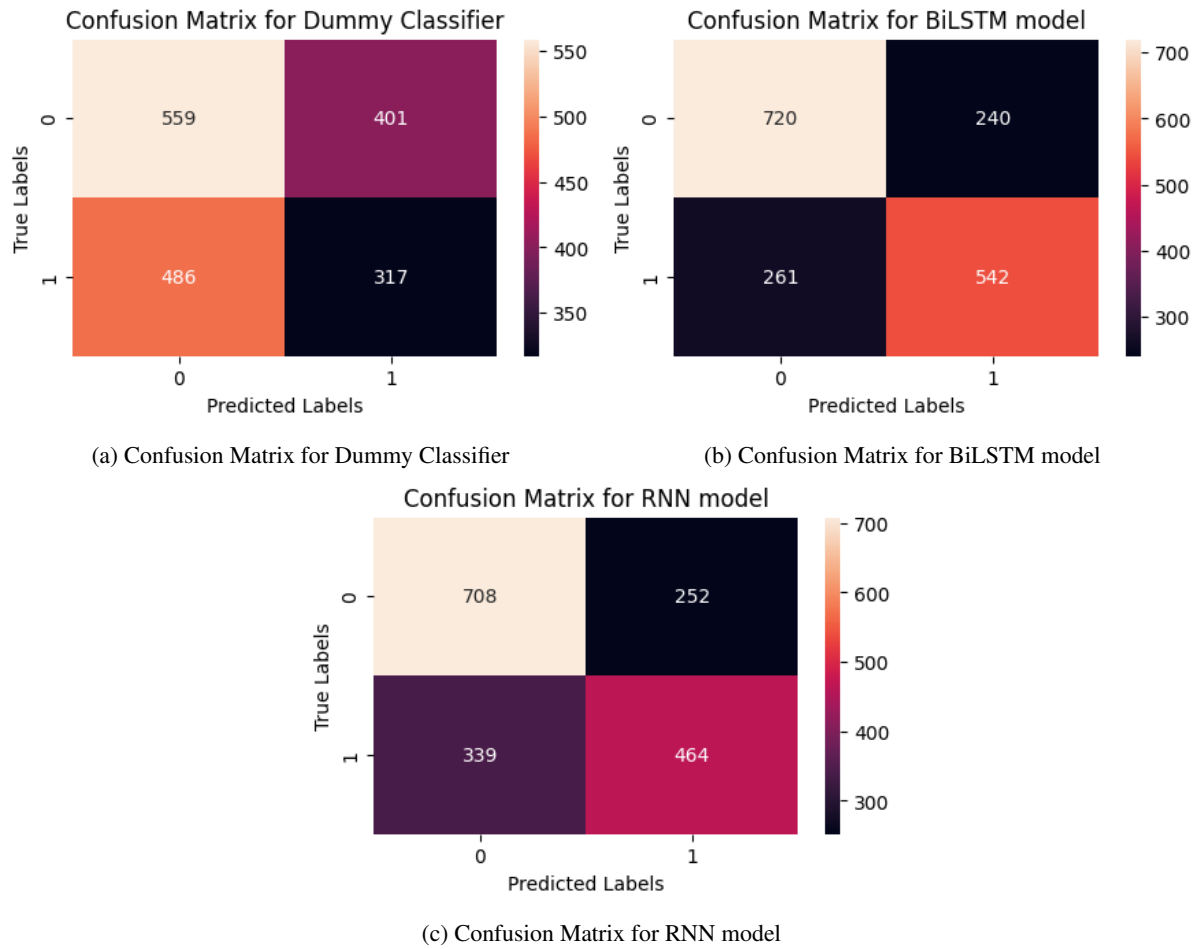


Figure 2: Models confusion matrices

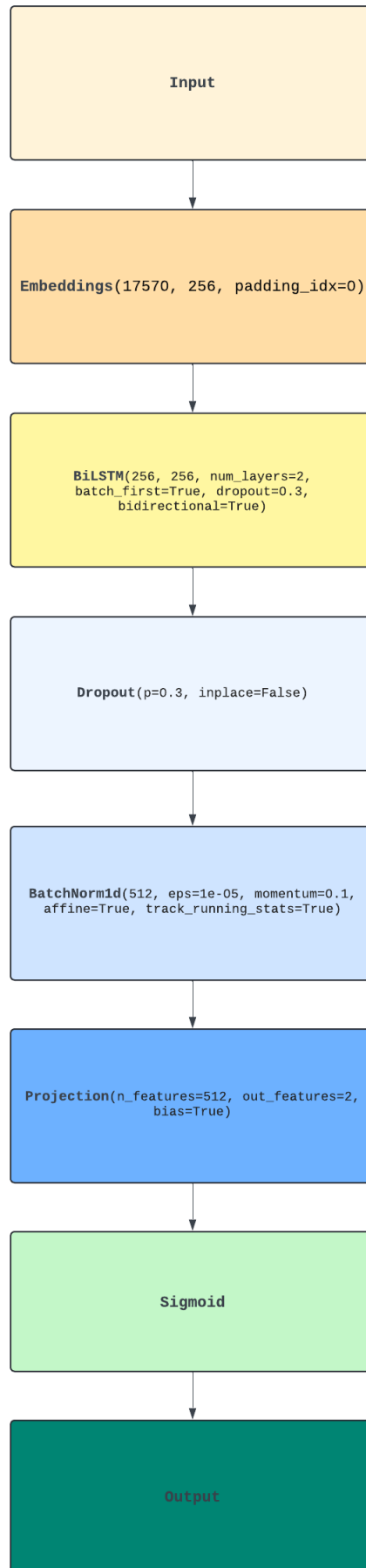


Figure 3: BiLSTM Flow Chart Architecture

[illegible]

Figure 4: Neutral vs Hate Speech

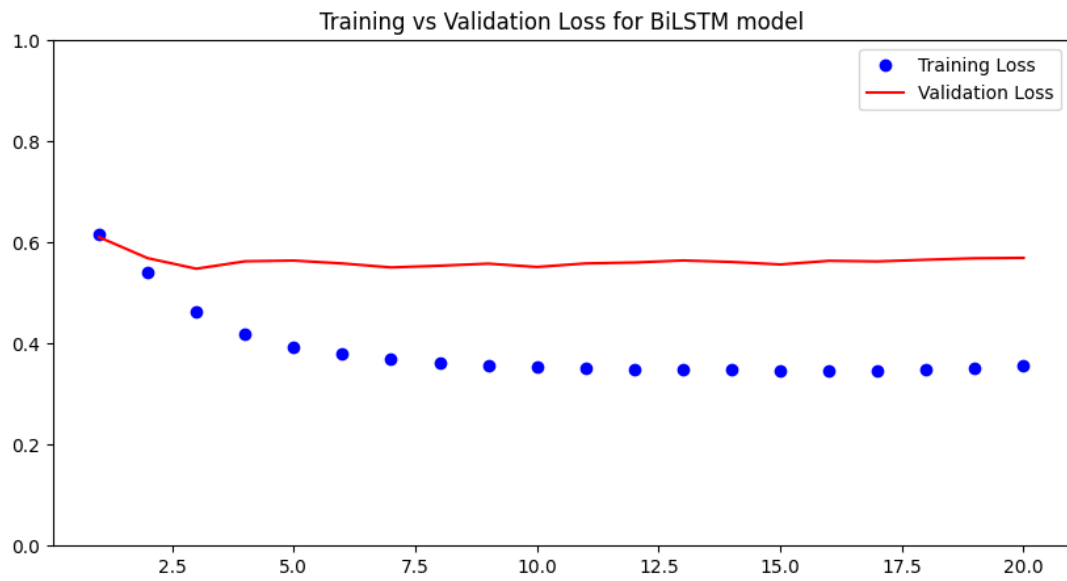


Figure 5: Train vs Validation Loss over epochs for BiLSTM model

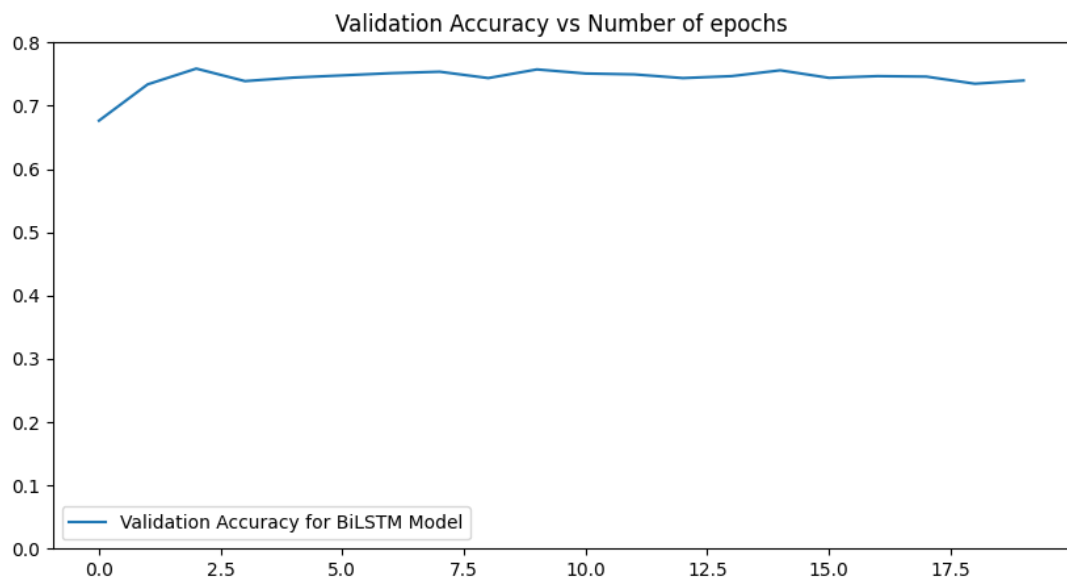


Figure 6: Validation Accuracy over epochs for BiLSTM model